

从 Tool Calling 到 Data Agent: 企业数据智能体的生产化路径

演讲人: 巴志欣

云器科技 AI产品总监

AiCon

全球人工智能开发与应用大会

极客邦科技 2026 年会议规划

促进软件开发及相关领域知识与创新的传播



参会咨询

InfoQ 技术大会

QCon 上海站

全球软件开发大会

大模型正在重新定义软件

Large Language Model Is Redefining
The Software

10月22-24日 / 上海建工浦江皇冠假日酒店

· AI Agent

· Vibe Coding

· 智能可观测

· 推理基建

· 模型攻防

· AI x 创造力

InfoQ 技术大会

AiCon 北京站

全球人工智能开发与应用大会

探索 AI 应用边界

Explore the limits of AI applications

12月18-19日 / 北京

· 大模型架构创新

· 多模态 AI 产业融合

· 具身智能

· AI for Science

· 大模型安全

目录

01 为什么：从Tool Calling 到为什么需要Data agent

02 演进：Data Agent演进路线

03 挑战：Data Agent的难点

04 破局：解决方案

05 实践：场景化案例

01

破题Why

通用Agent VS Data agent, 企业为什么需要更垂直的Agent

■ 通用 Agent 和 Data Agent, 差在哪

同样能碰数据, 方式完全不同



通用 Agent

主要靠工具调用 + Skill, 把数据平台当作外部工具来操作。

横向通用 · 什么都能干



Data Agent

聚焦数据领域的垂直智能体 —— 数据能力是原生的, 不是外接的。

垂直聚焦 · 为数据而生

一个「借工具够数据」, 一个「为数据而生」 —— 今天讲的, 是后者: **Data Agent**。

Data Agent

是针对企业数据环境做了深度领域化优化 —— 覆盖面更窄，却更准、更可信

维度	通用 Agent	Data Agent
主要目标	跨领域完成任务	从企业数据中查询、分析、产生洞察
工作环境	文件、代码、网页、API	数仓、湖仓、表、指标、报表、数据目录
核心上下文	当前对话、通用知识、工具说明	Schema、元数据、血缘、语义模型、业务口径、权限
典型动作	写代码、调 API、搜索、编辑文件	生成 / 执行 SQL、查表查文档、分析、排查数据问题
最大难点	任务规划和工具调用	找对数据、确认事实来源、处理指标歧义
治理要求	工具和系统级权限	行列级权限、脱敏、审计、数据新鲜度、合规边界

简单理解:通用 Agent 像「会用很多工具的项目助理」,Data Agent 像「懂数据资产、口径与权限的数据分析师」。

通用Agent VS Data Agent准确率对比

401 个真实数据任务评测 —— 数据原生的 Genie Code 同时赢了准确率和成本

为什么通用编程 Agent 不够(Databricks 观点)

1

数据发现规模巨大

要在大量表 / Notebook / Dashboard / 文档里找对资产。

2

判断 Source of Truth

不同表、报表、文档可能过时、矛盾、被替代。

3

缺少可验证测试

代码能单测,开放式业务问题却没有唯一标准答案。

401 任务评测 · 准确率 / 每任务成本

Genie Code 数据原生

76.6%

\$0.55

Coding Agent 1 通用编程

72.1%

\$1.09

Coding Agent 2 通用编程

55.9%

\$0.91

Coding Agent 3 通用编程

56.1%

\$1.16

结论 已理解数据语义与工作区上下文,少走「随机探索」路径 —— 同时拿到更高准确率、更低成本。

02

演进 Evolution

Data Agent的演进路线

Data Agent演进路线

不以「调用工具」为目的，以「业务价值」为目的。**Agent** 交付的是结果，不是动作。

阶段一

外部工具调用

MCP / CLI / API 接入系统能力

能把平台能力暴露出去，但看不到完整上下文，常常要回产品里确认。

问题：调用成功 $\neq$ 事情做完。
这是 **Tool Calling** 的位置。

阶段二

产品内置 Agent

结合页面状态、权限与业务对象

工具被大量使用，开始收集真实用户问题。通用工具编排仍缺业务闭环。

问题：会干活，但不会把一件事做完。
这是「内置」但未生产化。

阶段三

场景化 Data Agent

围绕开发 / 运维 / 质量做闭环

承接完整业务场景，交付可评估的结果。安全可控贯穿所有场景。

目标：从调用工具到完成业务目标。
这才进入生产化。

■ 阶段二被大量使用之后，三个战场自己浮现

场景与顾虑，都不是规划出来的，是用户反复问、反复怕，问出来的。

战场 · 数据开发

「单点到闭环」

要的不是一句 SQL，而是一整条加工链路。故事性、端到端。

一句话背后是口径、模型、任务、调度、资产。只生成 SQL，等于把最难的缝隙留给人。

战场 · 任务诊断

高频刚需

任务失败、耗时排查被问得最多。运维天天救火。

缺的不是又一个告警，是从证据到根因再到可落地修复的推理链。

战场 · 数据质量 DQC

治理刚需

质量问题反复，规则靠人工难维护。

治理不是一次性项目。规则建设要从老师傅经验，升级成持续运转的闭环。

贯穿所有战场的底线：安全与可控

客户最担忧的不是某个场景做不出来，是「它会不会把生产搞坏」。这不属于开发、运维或治理，而是企业级 Data Agent 的入场券：承接完整业务链路（闭环）+ 安全可控。

03

难点 Challenge

核心推理循环只占 1%,剩下 99% 的技术债,集中在这四道坎

Demo跑得很好，上生产就开始不可控

不是模型不够聪明。能上生产之前，这四道坎会反复出现

1 语义理解

如何精准理解业务语义，并在整个数据开发生产的链路中都可以保证口径的一致性和稳定性。

2 路径执行稳定

数据工程场景多样，目标是如何保障长对话 + 大数据结果 + 多工具 + 多轮迭代矛盾，进而做到在有限的记忆容量内，尽可能完整地处理用户的无限需求。

3 安全防护

生产场景上这么多运行中的任务和加工的表，如何保证不会误删除生产上的重要数据和任务

4 可靠性评估

每次版本迭代后，如何保证上线后的效果，包括工具调用、回答结果的准确率，回答耗时时长等

04

破局 Solution

四道坎,逐一给出可落地的破局路径

语义理解

→ 用 Semantic View 锚定业务口径

难点 · CHALLENGE

Agent 最大的问题不是“会不会回答”，而是能不能先找到正确的语义对象，再选择正确的执行路径，并且在整个过程中保持口径一致、权限可控、结果可验”。

所以真正的痛点不是单次问答，而是语义层全生命周期的生产化问题。

用户Case

- ✓ 场景 1：销售下滑归因 用户问：昨天华东区 GMV 为什么下降？
- ✓ 场景 2：指标口径统一 用户问：活跃用户、DAU、7日活跃分别怎么算
- ✓ 场景 3：新业务域落地 用户说：帮我把会员域的指标和维度建起来

机制

自然语言



Semantic View



Agent执行

语义层能力建设

原生 meta，并围绕其构建相应的产品能力

1、构建

- 多入口：Agent 创建 · 手动编辑 · API 集成
- 上传 BI 文件 (PowerBI / Tableau)，解析指标·维度·关系生成 meta

2、度量 & 优化

- 评估与追踪语义的使用效果
- 支持修改优化，让语义更精准

Semantic View DetailsInformationLineage版本更新记录

Tables & 字段关联

payments ← vw_effective_paymentsPK: order_id, payment_sequential有效支付明细表（已排除canceled/退款订单）

payment_type维度payments.payment_type支付渠道类型

gmv指标`sum`(payments.payment_value)渠道GMV（有效订单支付总额，已排除canceled/退款订单）

order_count指标`count`(DISTINCT payments.order_id)有效渠道订单量（已排除canceled/退款）

installment_order_count指标`count`(DISTINCT CASE WHEN payments.payment_installments > 1...分期付款订单数

orders ← olist_ordersPK: order_id订单主表

order_date维度CAST(orders.order_purchase_timestamp AS date)订单下单日期

order_month维度date_trunc('month', orders.order_purchase_timestamp)订单下单月份

order_year维度`year`(orders.order_purchase_timestamp)订单下单年份

order_status维度orders.order_status订单状态

canceled_order_count指标`count`(DISTINCT CASE WHEN orders.order_status = 'canceled' ...取消订单数

avg_delivery_days指标`avg`(datediff(orders.order_delivered_customer_date, orders...平均配送时长(天)

customers ← olist_customersPK: customer_id客户主表

customer_state维度customers.customer_state客户所在州

customer_city维度customers.customer_city客户所在城市

3、消费

- CLI · API 对接任意智能体，消费数据查询能力
- ETL 工程链路中，基于语义层创建加工链路

4、血缘追踪

- 可视化语义视图下游的全链路信息
- 一处修改，影响变更、同步更新

语义层的链路方案

1、基础能力&行为约束

Skill 路由开发链路下的多场景能力

- 创建 / 查询 / 调试 SV, 各走专门 skill。
- 不塞进一个 Prompt 自由推理, 场景不同路径不同。

统一工具约束

- 对 Semantic View 的增、删、改、查,全部必须通过工具执行,不靠模型自由发挥。
- 创建 / 修改 / 部署都走结构化接口, 每个动作对着定义。

2、语义理解

Discovery-First 系统提示里的硬规则

没指定 FQN 的分析性 SQL,先调 data-discovery 搜 SV。

双路径查询

找到SV, 按问题复杂度走两条路径

路径一·确定性执行

问题简单明确: 能直接映射到已知的 metrics + dimensions, 可以直接通过 SELECT ... FROM SEMANTIC_VIEW(...) 语句执行出来, 靠的是底层能力。

兜底方案

基于表、血缘, 上下文等裸写SQL

3、验证闭环

验证闭环

- 写入时做合法性校验, 结构不合法直接拦住。
- 兜底查询策略: 先语义 SQL、fallback 标准 SQL。

Semantic View 创建成功。来验证一下能否正常查询。

✦ 执行 2 工具 >

路径二·自主推理生成

问题比较复杂: 用户通过查询 semantic view 中定义的 metrics、dimensions、relationship、VQ 等信息, 自行判断并生成 SQL 语句。

■ 路径执行稳定

→ 上下文管理机制

难点 · CHALLENGE

数据工程任务 = 长对话 + 大数据结果 + 多工具 + 多轮迭代

用户Case

- 1、工具返回结果不可预测 一条SQL可能返回10行，也可能100000行，工具调用前对结果是不可预知的。
- 2、数据工程任务天然就是长对话 一个完整ETL产出会需要经历需求理解、模型设计、任务创建等
- 3、探索性任务需要读大量文件 用户说“帮我看看这个文件夹下任务之间的关系
- 4、系统提示本身就很大 有大量的内置工具和skill

机制

输入控制

→

运行时截断

→

上下文压缩

■ 上下文管理

第一层

输入控制

减少context的内容量

1 多Agent协同：先路由,再决定这一轮是探查、规划,还是执行。

- 探索：核查表结构、验证环境、调研配置 —— 返回快
- 规划：设计方案、拆解步骤、识别关键资源
- 执行：建表、写 SQL、建任务、配调度 —— 执行类脏活

2 工具按需加载、skill懒加载

只有触发时才会将内容加载到context中

第二层

运行时截断

防单次结果占满窗

1 工具结果截断（可再取）

- 单条正文超限：留头去中（或头+尾），中间丢掉。
- 整步多工具合计仍超：从最长的条继续收紧。

2 结果强制 limit + OSS

- 超过硬限不再尝试塞进消息：强制截断。
- 全文（日志、构建输出、大 JSON）上传 OSS。
- 窗里只留：URI、字节数、前几行摘要、如何再取。

第三层

上下文压缩

达到窗口阈值进行压缩

1. 清工具

越早轮次保留越少，近轮约 1500 字符、远轮约 300；当前轮完整保留,新轮次触发

2、压缩

压缩先判断是否达到窗口阈值，保留近 2 轮原文，剩余信息进行压缩。压缩信息总结为一条SystemMessage存档，保留关键信息：目标、动作、产物、决策、代办事项等

3、整轮截断

压缩后仍 $\geq 80\%$ ，对更早轮次强制截断，

安全防护

→ 三层机制严卡生产安全

难点 · CHALLENGE

写 / 删 / 大范围 / 敏感等高风险动作,只是通过关键字让LLM判断是无法做到精准控制的。模型的幻觉是无法保证执行约束

破局 · SOLUTION

- ✓ 沙箱隔离 兜底物理安全
- ✓ 系统提示约束行为边界 由模型先判定操作风险等级
- ✓ 工具拦截硬中断 用户明确批准后,才真正调用底层接口执行

机制

Agent 动作

→

沙箱隔离

→

系统约束

→

工具拦截

→

执行

安全防护： 示例， 任务删除

用户发起「删除」行为

系统进行行为判断，获取任务信息，并了解任务状态；

系统约束： 高风险行为拦截

强制弹窗触发用户确认

工具拦截

高危动作， 工具再次拦截确认

删除流程测试任务

我来查找"流程测试"相关的任务。

✦ 执行 1 工具 >

没有直接匹配"流程测试"的任务，可能是文件夹名称。让我查找名为"流程测试"的任务。

✦ 执行 1 工具 >

找到了"流程测试"文件夹（ID: 954010），让我查看里面的任务。

✦ 执行 1 工具 >

"流程测试"文件夹下有 3 个任务，均为草稿状态（未调度）：

❓ 用户输入

流程测试文件夹下有 3 个草稿任务，确认删除全部 3 个吗？还是只删除其中某些？

请输入回复...

提交

继续发送消息，Enter 进入队列等待，⌘ + Enter 引导当前回答

Skills bazh...n06 @ 68.7% 默认 停止

正在调用 delete_task 工具

允许 拒绝

继续发送消息，Enter 进入队列等待，⌘ + Enter 引导当前回答

Skills bazh...n06 @ 68.6% 默认 停止

■ 可靠性评估

→ 评测集 + 全链路 Trace + 灰度抽检

难点 · CHALLENGE

今天答对、明天答错,却发现不了、也量化不了 —— 质量变成一句「感觉还行」。

破局 · SOLUTION



「前」评测集回归 沉淀 golden set, 每次变更跑回归,守住已达到的质量。



「中」全链路 Trace 每步输入输出可观测、可回放,问题可定位到具体环节。



「后」反哺 把线上问题数据反哺到测试集中把“准不准”变成可量化指标。

机制

评测集



回归



Trace 观测



反哺



反馈闭环

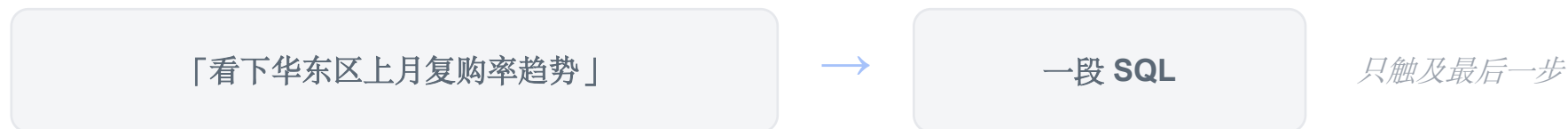
05

实践 Case

■ 场景一：数据链路开发

不是 text-to-SQL —— 一句话背后是一整条业务链路

text-to-SQL 的视角 · 止步于此



▽ 但一句话的背后，是一整条 ETL 业务过程 ▽



一句话 = 一整条 ETL 过程 —— **Agent** 要接住的是「链路」，不是「一句 SQL」。

场景一：数据链路开发

Agent 跑通「需求 → 语义 → 建模 → Pipeline → 上线」长链路

语义 沉淀为 Semantic View · 路径 多 Agent 规划编排 + 长链路上下文压缩 · 安全 上线前 HITL 确认 · 评估 上线后纳入评测回归

业务需求理解

基于用户话术理解需求并沉淀文档

语义沉淀

基于指标沉淀Semantic View

数仓建模

基于Semantic View规范数仓建模

Pipeline创建并验证上线

基于Semantic View规范数仓建模

Olist 巴西电商销售场景 — 指标需求文档

Olist 巴西电商销售场景 — 指标需求文档

1. 需求背景

- 业务背景：Olist 是巴西领先的电商平台 SaaS 服务商，连接中平台需要从订单、商品、支付、评价、客户、卖家和地理等多进行全面分析，以支撑运营决策、卖家管理和客户体验优化。
- 目标用户：平台运营团队、卖家管理团队、财务团队、客户体验
- 预期价值：建立统一指标口径，消除多部门数据歧义；支持日报、层月报、卖家绩效评估等场景

2. 数据源概览

GMV 语义视图已创建并验证通过。

Semantic View: brazilianecommerce.sv_gmv_analysis

建模架构 (5 张物理表 → 1 个语义视图)：

- items (olist_order_items) 作为核心粒度表，持有 GMV 指标
- 通过 FK 自动关联 orders → customers，以及 products、sellers
- 查询时引擎自动 JOIN，无需手写关联条件

3 个指标：

指标	说明
items.gmv	SUM(price + freight_value)，含运费，不
items.order_count	COUNT(DISTINCT order_id)，去重订单数
items.avg_order_value	gmv / order_count，客单价（派生指标）

Olist 巴西电商销售场景...

Olist GMV 数仓建模...

Olist 巴西电商 GMV 数仓建模规范设计

1. 概述

- 业务背景：基于 Olist 巴西电商数据集，构建 GMV（总交易额）分析数仓，支撑按时间、地区、商品类目、卖家、订单状态等多维度的销售分析
- 架构模式：Medallion (Bronze → Silver → Gold)
- 数据源：Lakehouse brazilianecommerce schema 下 8 张 olist_* 原始表
- 数据时间范围：2016-09-04 ~ 2018-10-17
- 更新策略：T+1 全量重算（静态数据集，无增量）

2. 分层架构图

计划进度 5/5

运行历史

21:05:42

已完成任务

运行成功 425ms

代码

日志

Step 0-2: 数据来源判断

阶段一确认 — 用户确认

查找/创建文件夹 99_baz

创建 4 个 SQL 任务 (DDI

保存 4 个任务的 SQL 内

序号	year	quarter	month
1	2,025	2	5
2	2,025	4	12
3	2,025	1	3

产物 6

报告

Olist 巴西电商销售场景 — 指标需求...

Olist GMV 数仓建模规范设计 — 阶段一

任务

DDL_建表脚本

ETL_Bronze层加载

ETL_Silver层加工

ETL_Gold层聚合

■ 场景二：从人工夜排到 Agent 自主诊断

场景 · 痛点

夜间调度作业失败,值班同学要翻日志、查血缘、逐层定位根因,平均耗时长、易漏、影响次日报表。

Agent 怎么做

- ✓ 自动拉取作业日志 + 数据血缘
- ✓ 定位失败根因,给出修复建议
- ✓ 高风险修复走 HITL 确认后执行
- ✓ 全链路 Trace 留痕,纳入评测回归

前 → 后(示例数据)

平均定位时长

45min → **8min**

一次性定位准确率

— → **85%+**

夜间人工介入

基线 → ↓ **60%**

小结 · TAKEAWAY

从 Tool Calling 到 Data Agent, 差的不是模型, 是这条 workflow 闭环。

三阶段（外部工具 → 内部 Agent → 场景化落地） × 三场景（开发 · 运维 · 质量）

同一条主线：从「单点工具调用」到「端到端 workflow 闭环」。

QCon

全球软件开发大会

上海
站

大模型正在重新定义软件

Large Language Model Is Redefining The Software

2026 年 10 月 22-24 日 / 上海建工浦江皇冠假日酒店



参会
咨询

**专题：从 App 到 Agent：字节跳动
Supabase 的 Agent-Native 规模化实践**

专题演讲嘉宾：张国强

字节跳动 / 资深技术专家



**专题：复杂业务 Agent 的持续进化：
快手电商导购的 Harness Loop 实践**

专题演讲嘉宾：包磊

快手 / 资深技术专家



**专题：从 Harness 到 Loop：阿福 Agent
小队如何处理持续涌入的线上 Badcase**

专题演讲嘉宾：肖汉松

蚂蚁集团 / 阿福 Harness 架构组负责人



**专题：AI Coding 在大型客户端工程中的
落地实践——从通才模型到专才 Agent**

专题演讲嘉宾：杨凯

小红书 / 客户端基础技术负责人



**专题：AI 时代的一人公司：从 Idea 到
Growth 的完整 workflow**

专题演讲嘉宾：casa

独立开发



**专题：Agent 驱动的全平台 GUI 持续
验证实践**

专题演讲嘉宾：王政达

微软 / SENIOR SOFTWARE ENGINEER



THANKS

探索 AI 应用边界

Explore the limits of AI applications

AiCon

全球人工智能开发与应用大会